FORMATION · JOUR 3

Agents IA autonomes

Module 03 · Concevoir des agents qui raisonnent, planifient et agissent

Bassem Ben Hamed

Autonomous AI Agents & RPA 2.0

Octobre 2026

Programme de la journée

Séquence	Contenu	Format
Rappel	Quiz du Jour 2, questions de la veille	
Module 3.1	Du LLM à l'agent autonome	
Module 3.2	Architecture d'un agent IA	
Atelier 3.A	Un agent ReAct sans framework	
Module 3.3	Outils, function calling et MCP	
Atelier 3.B	L'agent du service après-vente ★	
Module 3.4	Multi-agents, frameworks et observabilité	
Atelier 3.C	Une équipe d'agents	
Atelier 3.D	Observabilité et débogage ★	
Synthèse	Quiz et guide personnel de bonnes pratiques	

★ atelier qui fait avancer le projet fil rouge

Objectifs du jour

À la fin de la journée, vous serez capables de :

Développer un agent IA outillé, observable et doté de garde-fous

- choisir entre un workflow déterministe et un agent ;
- écrire la boucle d'un agent et décrire ses outils (function calling) ;
- exposer des outils métier par un serveur **MCP** ;
- construire un agent avec **LangGraph** : état, mémoire, validation humaine ;
- orchestrer plusieurs agents spécialisés ;
- tracer un agent et diagnostiquer ses pannes.

COMMENT C'EST ÉVALUÉ

Ateliers 3.A à 3.D ; livrable du projet fil rouge : agent du service après-vente instrumenté, dont les outils CRM passent par MCP.

Le fil rouge aujourd'hui



Le tri et l'assistant documentaire savent **répondre**. Aujourd'hui, l'assistant apprend à **agir** : consulter le CRM, calculer un remboursement, créer un ticket.

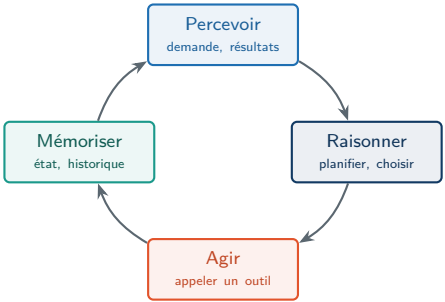
Dans notre projet

Livrable du jour : un agent LangGraph qui traite une demande de remboursement de bout en bout. Le CRM est exposé par un serveur MCP, le montant vient d'un calcul déterministe, la création du ticket attend la validation d'un opérateur, et chaque étape est tracée.

1

Du LLM à l'agent autonome

Qu'est-ce qu'un agent ?



Un **agent** est un système qui poursuit un objectif en enchaînant lui-même ses étapes : il observe, décide de la prochaine action, l'exécute avec un outil, observe le résultat, et recommence.

> En pratique : **un LLM qui utilise des outils en boucle**, avec une mémoire et des garde-fous, jusqu'à atteindre l'objectif ou une limite.

LLM, chatbot, assistant, agent

	LLM seul	Chatbot	Assistant (RAG)	Agent
Entrée	Un prompt	Une conversation	Une question	Un objectif
Accès au monde	Aucun	Aucun ou scénarisé	Documents	Outils : lire et **agir**
Qui fixe les étapes ?	Personne	Le concepteur	Le pipeline	**Le modèle**
Mémoire	Le prompt	La conversation	La conversation	État, historique
Risque	Erreur de texte	Erreur de texte	Erreur de texte	**Erreur d'action**

Dans notre projet

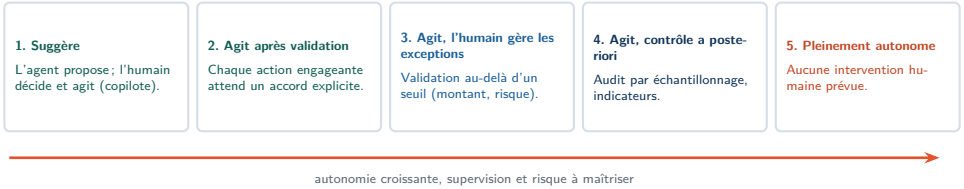
Jour 1 : un LLM trie et extrait. Jour 2 : un assistant répond à partir des documents. Jour 3 : un agent consulte le CRM, calcule, crée un ticket. Plus d'autonomie, donc plus de garde-fous.

Typologie des agents

- **Réactif** : réagit directement à ce qu'il perçoit, sans plan ; rapide, prévisible, limité.
- **Délibératif** : construit un plan avant d'agir et le révise ; adapté aux tâches longues.
- **Hybride** : réflexes pour les cas simples, délibération pour les autres ; c'est le cas de la plupart des agents à base de LLM.
- **Multi-agents** : plusieurs agents spécialisés coopèrent ; chacun son rôle, ses outils, son contexte.

> Ces catégories viennent de l'IA classique (agents à base de règles, de planification). Les agents à base de LLM les réunissent : le modèle peut réagir, planifier ou déléguer selon la tâche.

L'autonomie se règle



Le niveau se choisit **par action**, pas pour l'agent entier : consulter le CRM peut être autonome, rembourser 6 000 CHF ne l'est pas.

Dans notre projet

L'agent SAV de Fictiva : lectures autonomes (niveau 5), création de ticket validée par un opérateur (niveau 2). Au Jour 5, validation seulement au-delà d'un seuil (niveau 3).

Workflow ou agent ?

WORKFLOWS : LE CODE FIXE LES ÉTAPES

- **Chaînage** : extraire, puis vérifier, puis rédiger.
- **Routage** : un classement oriente vers le bon traitement.
- **Parallélisation** : plusieurs appels simultanés, puis synthèse.
- **Orchestrateur et exécutants** : un modèle découpe, d'autres exécutent.
- **Évaluateur-optimiseur** : produire, critiquer, corriger.

AGENTS : LE MODÈLE DÉCIDE

Utiles quand le chemin n'est pas connu d'avance : nombre et ordre des étapes variables, exploration, imprévus.

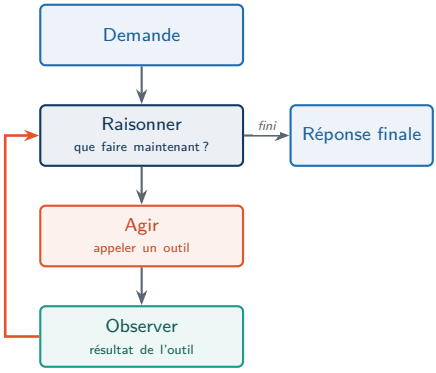
	Workflow	Agent
Prévisibilité	✓	⚠
Coût, latence	✓	⚠
Auditabilité	✓	⚠
Cas imprévus	✗	✓

Commencez par la solution la plus simple qui fonctionne, et n'ajoutez de l'autonomie que si la mesure le justifie. Catégories tirées de « Building effective agents » (Anthropic, 2024).

2

Architecture d'un agent IA

La boucle ReAct : raisonner, agir, observer



TRACE DE L'AGENT SAV (DEMANDE SAV-1)

```
appel  rechercher_client(CL-10456)
résultat Chevalier Conseil, standard
appel  calculer_remboursement(CMD-2026-0415,
[KB-80], defectueux=false)
résultat 216.0 CHF, validation non requise
appel  creer_ticket(CL-10456, remboursement, 216)
pause   validation de l'opérateur : oui
réponse Bonjour Madame Chevalier, ...
```

Le modèle n'a reçu aucun ordre d'étapes : il les a choisies d'après la méthode du prompt et les résultats obtenus.

ReAct textuel ou function calling ?

REACT TEXTUEL (2022)

```
Pensée : je dois connaître le taux CHF/EUR.
Action : taux_de_change[CHF, EUR]
Observation : 1.08
Pensée : je peux calculer...
```

Le programme analyse le texte pour trouver l'action : fragile (format, guillemets, hallucination d'observations).

FUNCTION CALLING (AUJOURD'HUI)

```
{"tool_calls": [{"name": "taux_de_change",
"arguments": {"devise_source": "CHF",
"devise_cible": "EUR"}}]}
```

L'API renvoie l'appel sous forme **structurée**, conforme au schéma de l'outil. Le raisonnement reste interne au modèle.

> Le principe ReAct (Yao et al., 2022) reste le même : alterner raisonnement et actions, chaque observation nourrissant la décision suivante. Les modèles de raisonnement et le function calling l'ont rendu fiable.

Planifier

Plan-and-execute

Un modèle écrit le plan complet ; l'exécution suit le plan, qui est révisé si un résultat le contredit.

Tâches longues, étapes nombreuses.

Décomposition

Découper l'objectif en sous-tâches indépendantes, éventuellement confiées à d'autres agents.

Demandes à plusieurs volets.

Réflexion

L'agent relit son résultat, le critique et le corrige avant de le livrer.

Rédaction, code, décisions à fort enjeu.

Avec un modèle de raisonnement, une planification légère suffit souvent : donnez l'objectif, la méthode et les critères de réussite dans le prompt, plutôt qu'un plan figé étape par étape.

La mémoire d'un agent

Mémoire	Contenu	Réalisation
Court terme	Messages, appels et résultats de la tâche en cours	La liste des messages, dans la fenêtre de contexte
État	Où en est la tâche, en pause ou non	Point de sauvegarde (*checkpointer*) par fil de conversation
Long terme	Connaissances durables	Base vectorielle (Jour 2), base de données
Épisodique	Dossiers déjà traités, préférences d'un client	Historique interrogeable, résumés

Gérer le contexte

À chaque tour, tout l'historique est renvoyé au modèle : les tokens augmentent. On résume les anciens échanges, on tronque les résultats d'outils trop longs, on ne transmet que ce qui sert à l'étape suivante.

Les garde-fous

LIMITER

- Nombre d'itérations, d'étapes du graphe.
- Budget de tokens, durée maximale.
- Nombre d'appels d'un même outil.

VÉRIFIER

- Arguments des outils validés par leur schéma.
- Sorties structurées, contrôles métier.
- Réponse vide ou malformée : relancer.

CONTRÔLER

- Validation humaine des actions engageantes.
- Outils en lecture seule par défaut.
- Journalisation de chaque étape.

> Une limite est un filet de sécurité, pas une solution : quand elle se déclenche, on corrige la conception (atelier 3.D).

Atelier 3.A · Un agent ReAct sans framework

À vous de jouer

Fichier : `ateliers/3A_agent_react.ipynb`

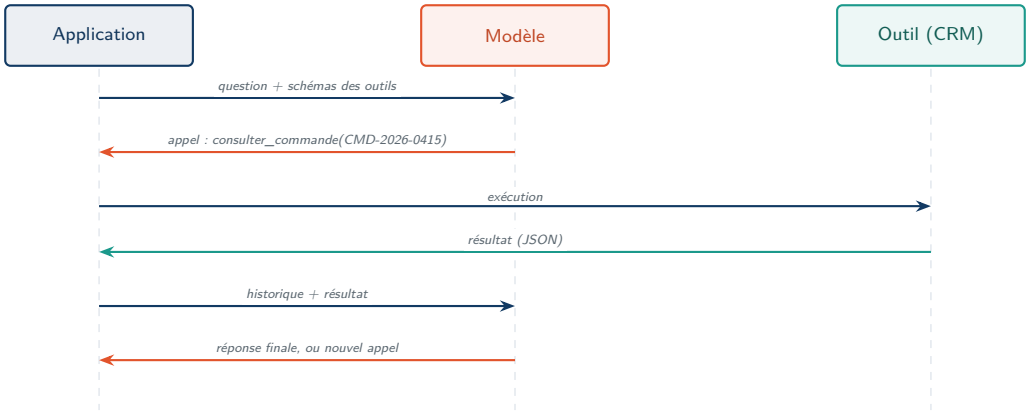
1. Écrire trois outils : calculatrice sûre, taux de change (BCE), recherche Wikipédia.
2. Les décrire au modèle (JSON Schema).
3. Écrire la boucle : appel du modèle, exécution des outils, retour des résultats.
4. Observer la planification, puis déclencher la limite d'itérations.
5. Ajouter un quatrième outil (météo, API Open-Meteo).

API publiques et gratuites, sans clé. Le modèle doit savoir appeler des outils : gpt-oss, Llama 3.3, Qwen3...

3

Outils, function calling et MCP

Le function calling pas à pas



Le modèle **n'exécute rien** : il demande. C'est l'application qui exécute, contrôle et décide de renvoyer le résultat.

Décrire un outil

```python
@tool
def calculer_remboursement(
        numero_commande: str,
        references: list[str],
        defectueux: bool,
        consommables_ouverts: bool = False) -> dict:
    """Calcule le montant remboursable pour des
    articles retournés, selon la politique de
    retours de Fictiva (délais, garantie, frais,
    exclusions, validation au-delà de 5 000 CHF).
    defectueux : true si le client signale un défaut.
    consommables_ouverts : true si des toners,
    cartouches ou du papier ont été ouverts."""
    return crm.calculer_remboursement(...)
```

Le décorateur produit le schéma envoyé au modèle :

- le **nom** de la fonction ;
- la **docstring**, qui devient la description ;
- les **types**, qui deviennent le JSON Schema des arguments.

Le modèle ne voit que cela : la description est un prompt.

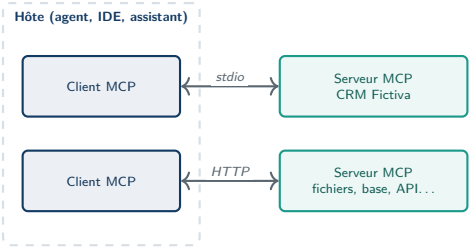
Bien concevoir ses outils

Principe	En pratique
Noms et descriptions explicites	`consulter_commande`, pas `outil_b` ; dire quand utiliser l'outil
Bonne granularité	Une action métier par outil ; ni trop fin (dix appels) ni trop large (fourre-tout)
Erreurs exploitables	« Commande introuvable. Format attendu : CMD-AAAA-NNNN » plutôt qu'une exception
Résultats concis	Renvoyer ce qui sert à la décision, pas une page entière
Calculs déterministes	Montants, délais, règles : en code, testés, jamais « de tête » par le modèle
Écriture idempotente	Un appel répété ne crée pas deux tickets
Jamais de demande de relance	Un outil ne dit pas « rappelez-moi » : risque de boucle

Dans notre projet

Atelier 3.D : avec trois outils nommés « Outil du CRM », l'agent tâtonne ; avec des descriptions précises, il enchaîne les bons appels.

MCP : un standard pour brancher les outils



Le **Model Context Protocol** est un standard ouvert, lancé par Anthropic fin 2024 et adopté par les principaux fournisseurs et outils.

- Un serveur expose des **outils**, des **ressources** (données à lire) et des **prompts**.
- Transports : `stdio` en local, HTTP pour un serveur distant.
- Écrit une fois, un serveur sert n'importe quel client compatible.

MCP relie un agent à ses **outils et données**. Pour relier des agents entre eux, d'autres protocoles existent, comme A2A (module 3.4).

Un serveur MCP en quelques lignes

```python
from mcp.server.fastmcp import FastMCP
import crm

serveur = FastMCP("crm-fictiva")

@serveur.tool()
def consulter_commande(numero_commande: str) -> dict:
    """Détail d'une commande (format CMD-AAAA-NNNN) :
    client et statut, dates, lignes, montants."""
    return crm.consulter_commande(numero_commande)

if __name__ == "__main__":
    serveur.run()          # transport stdio
```

Côté agent :

```python
client = MultiServerMCPClient({"crm": {
    "command": "python",
    "args": ["serveur_crm_mcp.py"],
    "transport": "stdio"}})
outils = await client.get_tools()
```

Les outils MCP deviennent des outils LangChain comme les autres.

Dans notre projet

Le serveur du CRM n'expose que trois outils **en lecture** : rechercher un client, lister ses commandes, consulter une commande.

Connecter les systèmes de l'entreprise

Source	Outil typique
API REST	Appel HTTP encapsulé, réponse résumée
Base de données	Requête paramétrée, en lecture seule
CRM, ERP	Consultation, puis écriture validée
Documents	Recherche RAG (Jour 2)
Messagerie	Brouillon, jamais envoi direct

ENCAPSULER, TOUJOURS

- Jamais de SQL ou d'URL libre écrit par le modèle : des paramètres validés.
- Authentification gérée par l'outil, pas par le prompt.
- Réponses filtrées : seulement les champs utiles et autorisés.
- Chaque appel journalisé.

Sécurité des outils

- **Moindre privilège** : chaque agent ne reçoit que les outils de sa mission, en lecture seule par défaut.
- **Validation humaine** des actions engageantes : écrire, payer, envoyer, supprimer.
- **Bac à sable** pour exécuter du code : conteneur isolé, sans réseau ni secrets.
- **Secrets** hors du prompt : coffre-fort, OAuth, jetons à durée limitée.
- **Résultats d'outils non fiables** : un email ou une page web peut contenir des instructions (injection indirecte, Jour 4).
- **Traçabilité** : qui a demandé quoi, quel outil, quelle validation.

L'« autonomie excessive » (*excessive agency*) figure dans le Top 10 OWASP des risques des applications LLM : trop d'outils, trop de droits, pas assez de validation.

Atelier 3.B · L'agent du service après-vente ★

À vous de jouer

Fichier : `ateliers/3B_agent_sav.ipynb`

1. Explorer le CRM simulé de Fictiva.
2. Lancer le serveur MCP du CRM, lister et appeler ses outils.
3. Construire l'agent LangGraph : modèle, outils MCP et locaux, mémoire.
4. Traiter une demande ; valider le ticket comme opérateur.
5. Refuser une proposition erronée (commande non livrée).
6. Traiter toutes les demandes ; comparer les montants aux montants attendus.

Dans notre projet

Livrable : l'agent du service après-vente, réutilisé au Jour 4 avec l'assistant documentaire.

4

Multi-agents, frameworks et observabilité

Plusieurs agents : quand et comment



POURQUOI

Rôles spécialisés, chacun avec son prompt et ses seuls outils ; contextes séparés ; tâches parallèles ; regards croisés.

PRIX À PAYER

Plus d'appels et de tokens, coordination délicate, pannes plus difficiles à diagnostiquer. Un agent unique bien conçu d'abord.

État partagé et protocoles

DANS UNE MÊME APPLICATION

Les agents partagent un **état** : demande, fiche d'analyse, brouillon, avis du relecteur. Un graphe décide qui intervient ensuite.

Dans notre projet

Atelier 3.C : analyste, rédacteur et relecteur partagent un état ; le relecteur renvoie au rédacteur, deux fois au plus.

ENTRE APPLICATIONS : A2A

Le protocole **Agent2Agent**, lancé par Google en 2025 et confié à la Linux Foundation, permet à des agents de fournisseurs différents de coopérer :

- chaque agent publie une « carte » de ses compétences ;
- les agents s'échangent des tâches et leurs résultats.

MCP : agent → outils. A2A : agent → agent.

Panorama des frameworks

Framework	Approche
LangGraph	Graphes d'état explicites : contrôle fin, mémoire, reprise, validation humaine
OpenAI Agents SDK	Agents, passages de relais, garde-fous et traces intégrés
Claude Agent SDK	Le harnais de Claude Code : outils de fichiers, terminal, sous-agents
Google ADK	Kit de développement d'agents de Google, multi-agents
Microsoft Agent Framework	Successeur d'AutoGen et de Semantic Kernel
CrewAI	Équipes d'agents définies par rôles et tâches
Pydantic AI	Agents typés, sorties validées par Pydantic
smolagents	Agents qui écrivent et exécutent du code Python

CRITÈRES DE CHOIX

Niveau de contrôle, compatibilité avec plusieurs fournisseurs et les modèles open-weight, validation humaine, observabilité, maturité. L'écosystème bouge vite : les concepts (boucle, outils, état, garde-fous) restent.

Observabilité



Trace illustrative

Une **trace** enregistre chaque étape : appels du modèle et des outils, entrées, sorties, tokens, durée, coût, erreurs.

- **Langfuse** : open source, auto-hébergeable, offre cloud gratuite.
- **LangSmith** : plateforme de LangChain.
- **Arize Phoenix** : open source, local.
- Standard sous-jacent : OpenTelemetry et ses conventions pour l'IA générative.

Au-delà du débogage : évaluer les **trajectoires** (bons outils, nombre d'étapes) et suivre les coûts.

Exemples d'architectures

Agent	Outils	Risque principal	Garde-fou
Recherche	Web, documents, synthèse	Sources peu fiables	Citations, sources autorisées
Code	Fichiers, terminal, tests	Commandes destructrices	Bac à sable, revue humaine
Commercial	CRM, catalogue, devis	Promesses non autorisées	Plafonds, validation
Support	CRM, documentation, tickets	Remboursement erroné	Calcul déterministe, validation

Même architecture partout : un modèle, des outils, un état, des garde-fous. La différence tient aux outils autorisés et au niveau d'autonomie accordé à chaque action.

Atelier 3.C · Une équipe d'agents

À vous de jouer

Fichier : `ateliers/3C_multi_agents.ipynb`

1. Définir trois rôles : analyste (outils CRM), rédacteur, relecteur (verdict structuré).
2. Les relier dans un graphe LangGraph à état partagé, avec une boucle de correction limitée.
3. Traiter une demande et lire la fiche, les versions et les revues.
4. Comparer avec un agent unique : qualité, appels, tokens, durée.

Atelier 3.D · Observabilité et débogage ★

À vous de jouer

Fichier : `ateliers/3D_observabilite.ipynb`

1. Tracer l'agent du service après-vente : traceur local, et Langfuse si configuré.
2. Panne n° 1 : un outil qui plante ; erreurs exploitables.
3. Panne n° 2 : des outils mal décrits ; comparer les traces.
4. Panne n° 3 : une boucle sans fin ; limite d'étapes, puis correction de l'outil.

Dans notre projet

Livrable : l'agent du service après-vente instrumenté. Langfuse est facultatif : offre cloud gratuite ou instance auto-hébergée.

5

Synthèse

À retenir

1. Un agent est un **LLM qui utilise des outils en boucle**, avec une mémoire et des garde-fous.
2. Workflow d'abord ; agent quand le chemin ne peut pas être fixé à l'avance.
3. Le modèle **demande**, l'application **exécute** : calculs et règles restent dans le code.
4. Les noms et descriptions des outils sont un prompt ; les erreurs doivent être exploitables.
5. **MCP** standardise l'accès aux outils ; moindre privilège et validation humaine encadrent l'action.
6. Plusieurs agents seulement si la mesure le justifie, toujours avec une limite de tours.
7. Pas d'agent sans **trace** : la plupart des pannes viennent des outils.

Quiz

1. Qu'est-ce qui distingue un agent d'un assistant RAG ?
2. Citez deux critères qui font préférer un workflow à un agent.
3. Que voit exactement le modèle d'un outil ?
4. Pourquoi le montant d'un remboursement doit-il être calculé par un outil ?
5. Quels sont les rôles du client et du serveur dans MCP ?
6. Que se passe-t-il si un outil répond « réessayez plus tard » ?
7. Quel est l'intérêt d'un relecteur dans une équipe d'agents, et son coût ?
8. Que doit contenir une trace d'agent pour être utile ?

GUIDE PERSONNEL

Ajoutez vos cinq bonnes pratiques sur les agents à celles des Jours 1 et 2.

Demain : agents RAG

L'agent consulte le CRM, mais il ne connaît pas la documentation ; l'assistant du Jour 2 connaît la documentation, mais ne raisonne pas sur plusieurs étapes.

Demain, on les réunit :

- le RAG devient un outil de l'agent ;
- routage entre plusieurs sources, auto-correction ;
- évaluation des trajectoires et tests d'injection.

